

Devbot — API Usage (no GUI)

Once the package is installed, you can use the library directly in your code. Below is a compact reference for saving API keys, listing supported models, and invoking the core LLM call.

1) Save your API key(s)

Before querying a model, save the corresponding provider key to file.

Code:

```
from devbot_sa.devbot_assistant import store_api_key_to_file

# provider: "openai" | "google" | "groq"

store_api_key_to_file("openai", "<YOUR_OPENAI_API_KEY>")

store_api_key_to_file("google", "<YOUR_GOOGLE_API_KEY>")

store_api_key_to_file("groq", "<YOUR_GROQ_API_KEY>")
```

What it does

- Writes the key into a file next to the library module:
 - openai_api_key, google_api_key, or groq_api_key.
- These files are read automatically when you call `query_llm(...)`.
- This method must be called with your api key and the relevant provider (use the same name for the providers as shown above) before you call **query_llm** function.

2) See which models are available

Code:

```
from devbot_sa.devbot_assistant import available_models

models = available_models()

print(models)
```

What it does:

- Returns a list[str] of model identifiers the library can query:
 - OpenAI (e.g., gpt-4o, gpt-4o-mini, o3, o1, gpt-5, etc.)
 - Google Gemini (e.g., gemini-2.0-flash, gemini-2.5-pro, gemini-2.5-flash)
 - Groq / Llama variants (e.g., llama-3.3-70b-versatile, ...)
- Choose a model that matches the provider whose key you saved.
- The model name you pass to the **query_llm** should be exactly same as returned from this function.

3) Call the core LLM API

Code:

```
from devbot_sa.devbot_assistant import query_llm

reply_text = query_llm(
    model="gpt-4o",          # required model name (string)
    human_message="Design a high-level architecture for a note-taking app
using PlantUML.",
    continue_chat=False,    # False for first turn; True to continue from
last run
    system_message=(
        "You are a software architecture assistant named 'Devbot'. "
        "Only discuss software architecture and use PlantUML for diagrams."
    ),
    temperature=0.3,        # optional float; defaults to provider default if None
)

print(reply_text) # The output returned by the model
```

Inputs

- **model:** str
Model identifier (must be one from `available_models()`).
- **human_message:** str
The **exact prompt** you want sent as the human message.
- **continue_chat:** bool = False
If True, the function will read the previous reply from the output file (saved next to the module) and include it as the prior AI message for context. Otherwise sends only the prompt without any chat history.
- **system_message:** Optional[str] = None
Custom system instruction. If None, an empty string is used (you can pass your own or reuse the project's `SYSTEM_PROMPT`).
- **temperature:** Optional[float] = None
Sampling temperature; if None, the model's default is used.

Return value

- **str:** the model's reply text.